Stochastic Simulation Assignment 1
Module 8

Tycho Borm Mark Tijhaar

May 12, 2026

1 Introduction

We are asked to simulate a simplified version of a limit order book for a single financial asset (e.g. stocks, crypto currency, etc.). This consists of buy orders and sell orders, which want to buy this asset for a maximum price and sell for a minimum price respectively. This will be simulated until the 500^{th} trade is made.

The orders have deterministic inter-arrival times between the n^{th} and $(n-1)^{\text{th}}$ order defined by

$$\Delta t_n = 5\left(1 + \left|\sin\left(\frac{n\pi}{3}\right)\right|\right) \quad \text{(seconds)}$$

The chance for being a buy or sell order is both 0.5. Additionally, there is a 0.3 probability that the order is a market order, which will immediately buy or sell for the best price. If it is not a market order the order will get a value determined by

$$p_n = 100 + 2\sin(ne) \quad \text{(rounded at two decimals)}$$

A trade is sufficient and therefore will be made if best bid $\geq$ best ask. If there is no immediate sufficient trade for the order, it will be saved in a list, in case later on there will be. But an event is cancelled if after a certain time it still has not been matched, determined by

$$\tau_n = t_{\text{arrival},n} + 30\left(1 + \cos^2(n)\right) \quad \text{(seconds)}$$

at arrival.

Our code can be found in the github repository in the file LimitOrderBook.py

2 Method

To run the simulation we used a total of five classes, of which two inherit the event class that was provided in the core module.
The biggest class was the MatchingEngine class, this class is used not only to perform matches, but also to keep track of both queues, and keep track of statistics. We made the choice to put all of this in a single class, because we don't want to pass multiple objects into every function, since both events need all three of these things and code can become very clutterd if every class has a field for matching engine, statistics, and both queues. The matching engine creates a statistics class object and manages all statistics, since everything that happens with cancelled and accepted orders and adding and removing things to the queue happens within it's methods. MatchingEngine has methods to add/remove from both queues while updating statistics, generating and handeling a market order, finding matches between queues, and handeling an accepted order. Since accepting orders happens in two different places we made it a separate method to avoid code duplication.
The Statistics is a wrapper class that just holds all TimeAveragedStatistic and SampleStatistic objects for all statistics. In addition to this the bid-ask spread statistic is kept in a list and calculated at the end, since as far as we know the TimeAveragedStatistic class does not support the fact that the spread does not get counted while one of the queues is empty. Our implementation works the exact same way as the given one, but it keeps track of a total elapsed time variable that we divide by when asking the mean so we can just skip any times when one of the queues is empty while still getting a correct result. It also has a method to print all statistics which we call after finishing the simulation.
The LimitOrder class is just a datastructure with fields for it's price, a bool for if it is a bid or

ask order, it's arrival time, to calculate average time-to-fill and a reference to it's own cancel event, so the matching engine can cancel it's cancel event when accepting an order.
The OrderArrival class inherits from the given Event class, when it get's executed it generates a new Order, and a new CancelEvent if that order is a limit order. It hands this order to the MatchingEngine Entity and then asks for a match check.
The CancelEvent class also inherits from the given Event class, when it get's executed it tells the matching engine entity to remove the corresponding limit order from it's queue.
When the simulation is started, a MatchingEngine is created, and an initial orderarrival is scheduled which is passed the MatchingEngine. Then the simulation is ran, and afterwards the statistics are printed.
After initialization, running the simulation is described by Algorithm 1

Algorithm 1 LimitOrderBook simulation

while Simulation queue not empty and simulation not terminated **do**
  $E \leftarrow$ next event scheduled
  if E is OrderArrival **then**
    Generate a new order and handle it according to Algorithm 2
  else if E is CancelEvent **then**
    Remove the corresponding order from the queueu
    Update queue statistics
    Update spread statistics
    Updade cancel statistics
    Attempt match according to Algorithm 3
  end if
end while
Calculate cancel rate
Calculate spread statistic
Return all statistics

Algorithm 2 Generate new order

if Order is a limit order with change 0.7 **then**
  determine if order is a buy or sell order
  Schedule a new OrderArrival according to specifications
  Schedule corresponding CancelEvent
  Update queue statistics
  Update spread statistics
  Attempt match according to Algorithm 3
else if Order is a market order **then**
  determine if order is buy or sell order
  match it with the best ask/bid price in the queue
  accept this order and remove the corresponding cancel event
  Update queue statistics
  Update spread statistics
  Update time-to-fill statistics
  Update total filled orders
end if

Algorithm 3 Attempt match

```
if Highest bid price ≥ Lowest ask price then
    Match is made, remove both from their queues
    Remove both of their cancel events
    Update queue statistics
    Update spread statistics
    Update time-to-fill statistics
    Update total filled orders
    if Total filled orders ≥ 500 then
        Terminate simulation
    end if
end if
```

3 Data collection

The Time-averaged bid-ask spread is updated after any of the queues changes in length. It takes the highest buy order and the lowest sell order if both lists are non-empty. It also keeps in account for how long this is true. If one or both of the lists are empty, it cannot be computed and the time is also not taken into account. After the simulation stops the average is computed.

The Average time-to-fill is computed if a buy order and sell order match. For each of them the time of arrival of the offer is subtracted from the and the matching time (departure time) of the offer and gets saved. At the end this is averaged out.

The Cancellation rate is computed by dividing the number of cancelled orders by the number of cancelled orders plus the number of successful orders. The number cancelled orders is updated when an order is cancelled in the matching engine entity and the number of successful orders is updated after a trade.

The Average ask and bid queue lengths get updated after the respective queue gets updated. Then it looks at the last time that queue was updated and the queue length it then changed to. The old queue length is multiplied by the elapsed time (current time - time of the previous update) and added by the Average ask or bid queue length which starts at 0. Then the other two number above gets updated. When the simulation ends, this is done for one last time and then the Average queue lengths gets divided by the total elapsed time.

4 Results

We ran the simulation with the set random seed 67 for testing purpose with calculating the statistics, but also tested with random seeds to see what happens on average and what happens when running the simulation with a higher horizon. After running the simulation we obtained the following results.

Metric	Value
Time-averaged bid-ask spread	1.80
Average time-to-fill	13.28
Cancellation rate	0.30
Average ask queue length	0.93
Average bid queue length	1.06

Table 1: Limit order book Metrics

A Time-averaged bid-ask spread of roughly 1.80 indicates a fairly liquid market. This is because the highest buy and lowest sell order is relatively close to each other, since the mean price is around 100.
An Average time-to-fill of 13.28 is relatively small, since the inter-arrival times are somewhere between 5 and 10. This means an order has to wait for two new orders on average before the trade happens.
A Cancellation rate of 0.30 is also relatively low, since in a market where everyone is trying to make money, there is going to be alot of buy/sell offers that ask an unreasonable price to make more profit. The low rate is likely due to the high availability of market orders, which trade orders quickly and prevent large queues
The Average queue lengths are close to each other and tend to be somewhere around 0.95. This means there roughly as much supply and demand.
When running the simulation for longer periods most these metrics stay relatively close to these values, and the average queue lengths get closer to each other, since in the limit they should be the same. This is because the chance for a buy/sell limit or market order is a 50/50.